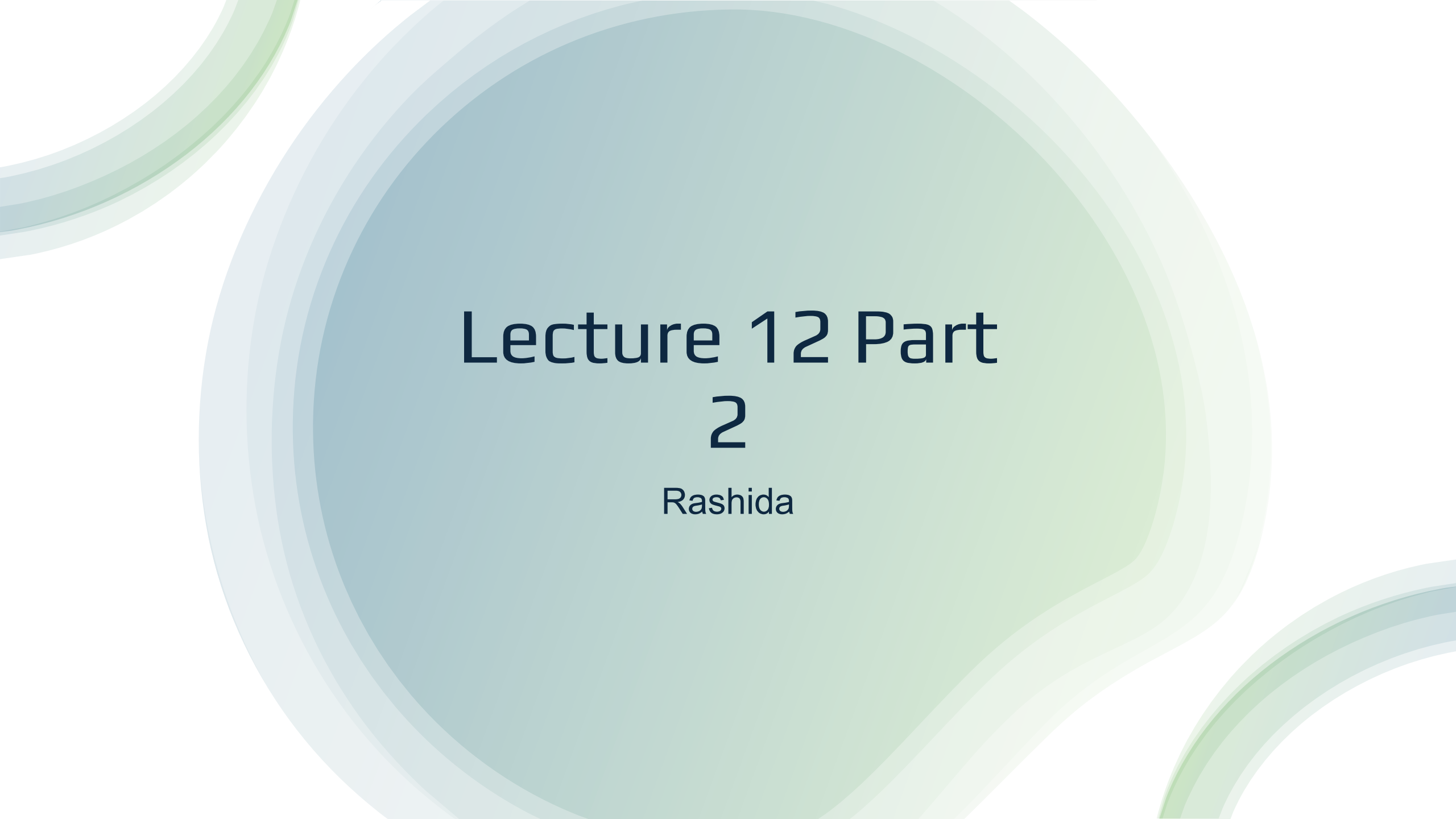


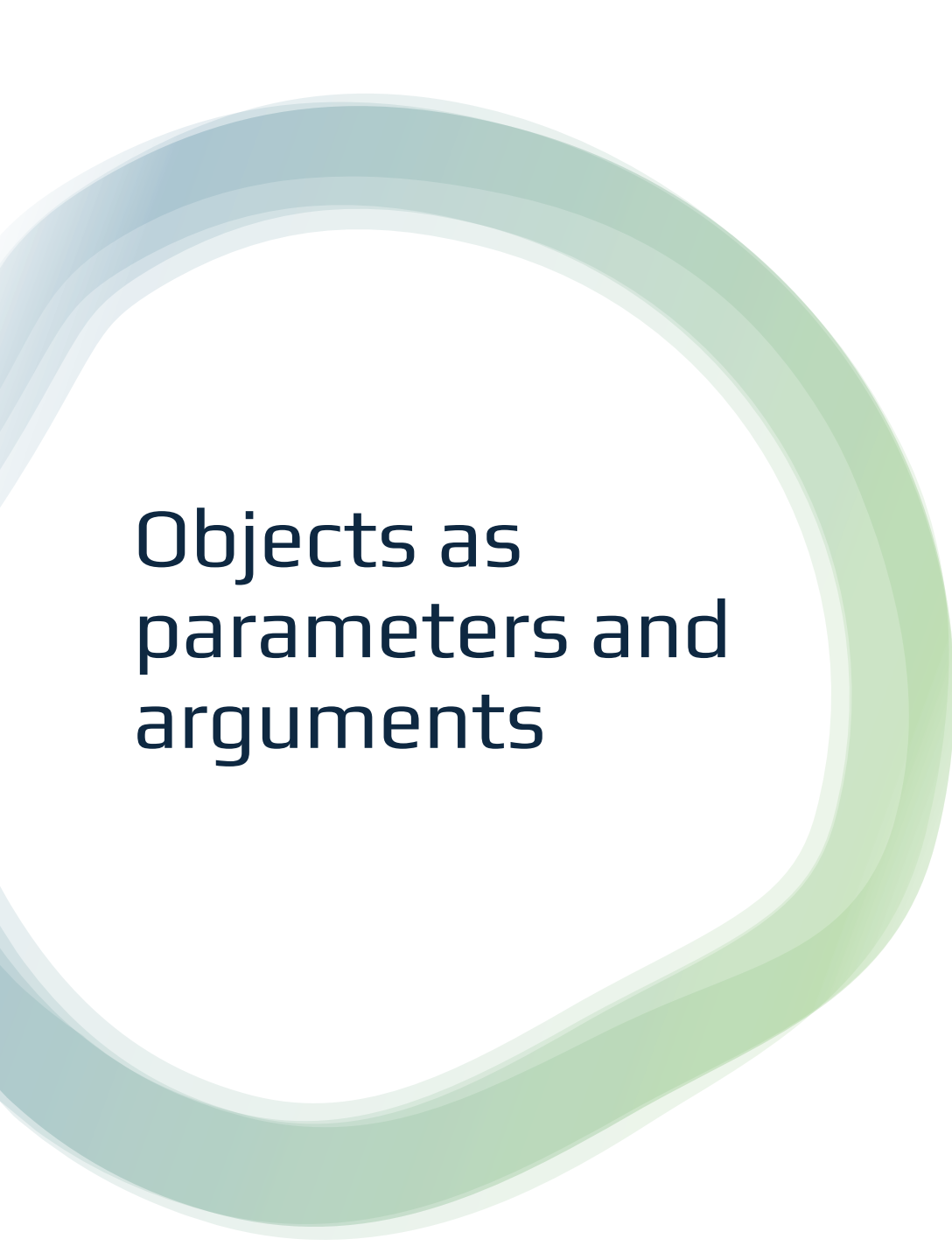


# Lecture 12 Part 2

Rashida

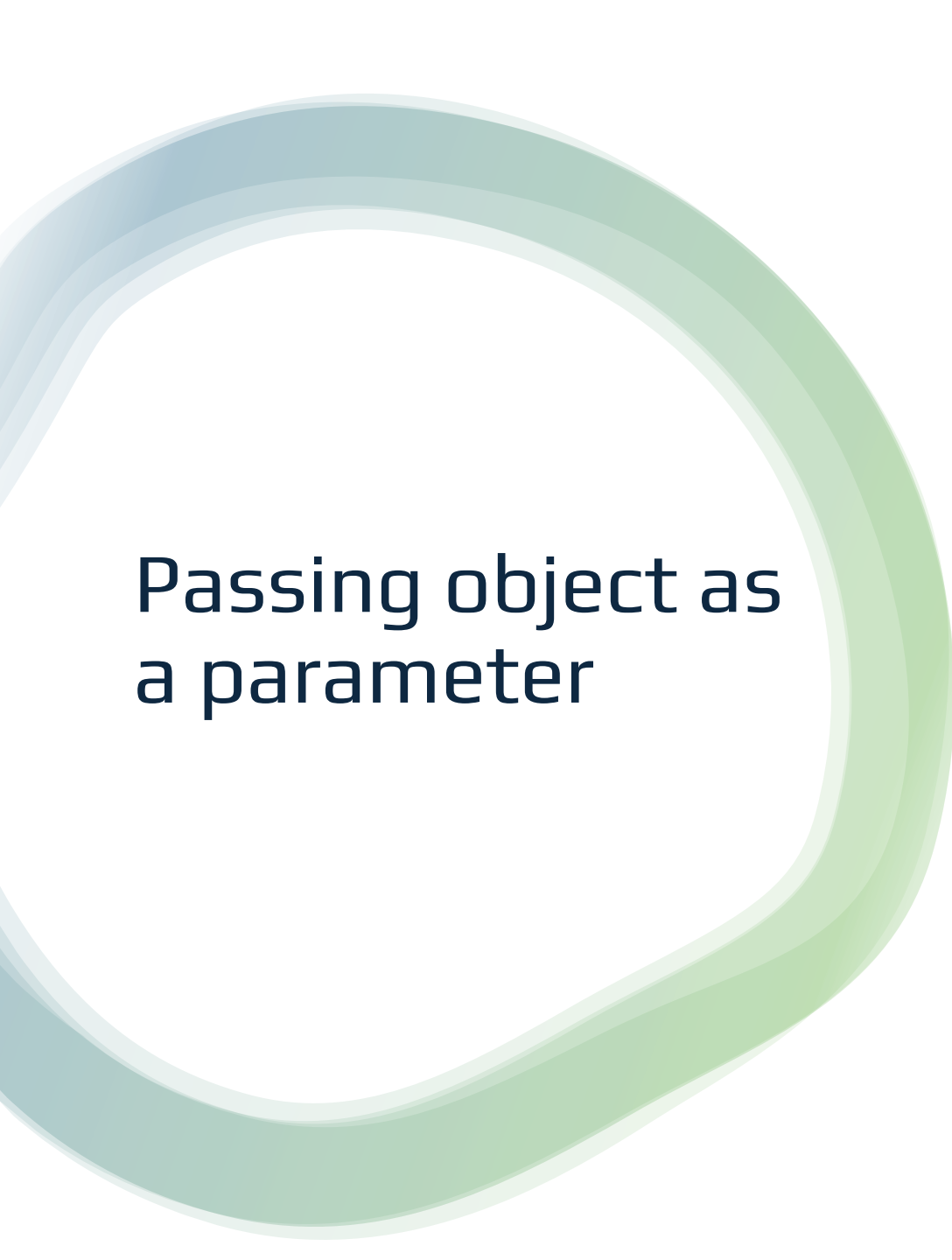
# Agenda

- **Overloading methods**
- **Overloading constructors**
- **Parameters and arguments**
- **Pass by value**
- **Pass by reference**
- **Objects as parameters and arguments**



# Objects as parameters and arguments

- In Java, objects can be passed to methods just like primitive data types. This is a core feature of object-oriented programming that allows methods to access and manipulate an object's data.
- Java is strictly **pass-by-value**. However, for objects, the "value" being passed is the **reference** (memory address) to the actual object, not the object itself. This leads to an effect similar to pass-by-reference.



## Passing object as a parameter

- In Java, When a primitive type is passed to a method ,it is done by use of call-by-value . Objects are implicitly passed by use of call-by-reference.
- This means when we pass primitive data types to method it will pass only values to function parameters so any change made in parameter will not affect the value of actual parameters.
- Whereas Objects in java are reference variables, so for objects a value which is the reference to the object is passed. Hence the whole object is not passed but its referenced gets passed. All modification to the object in the method would modify the object in the Heap.

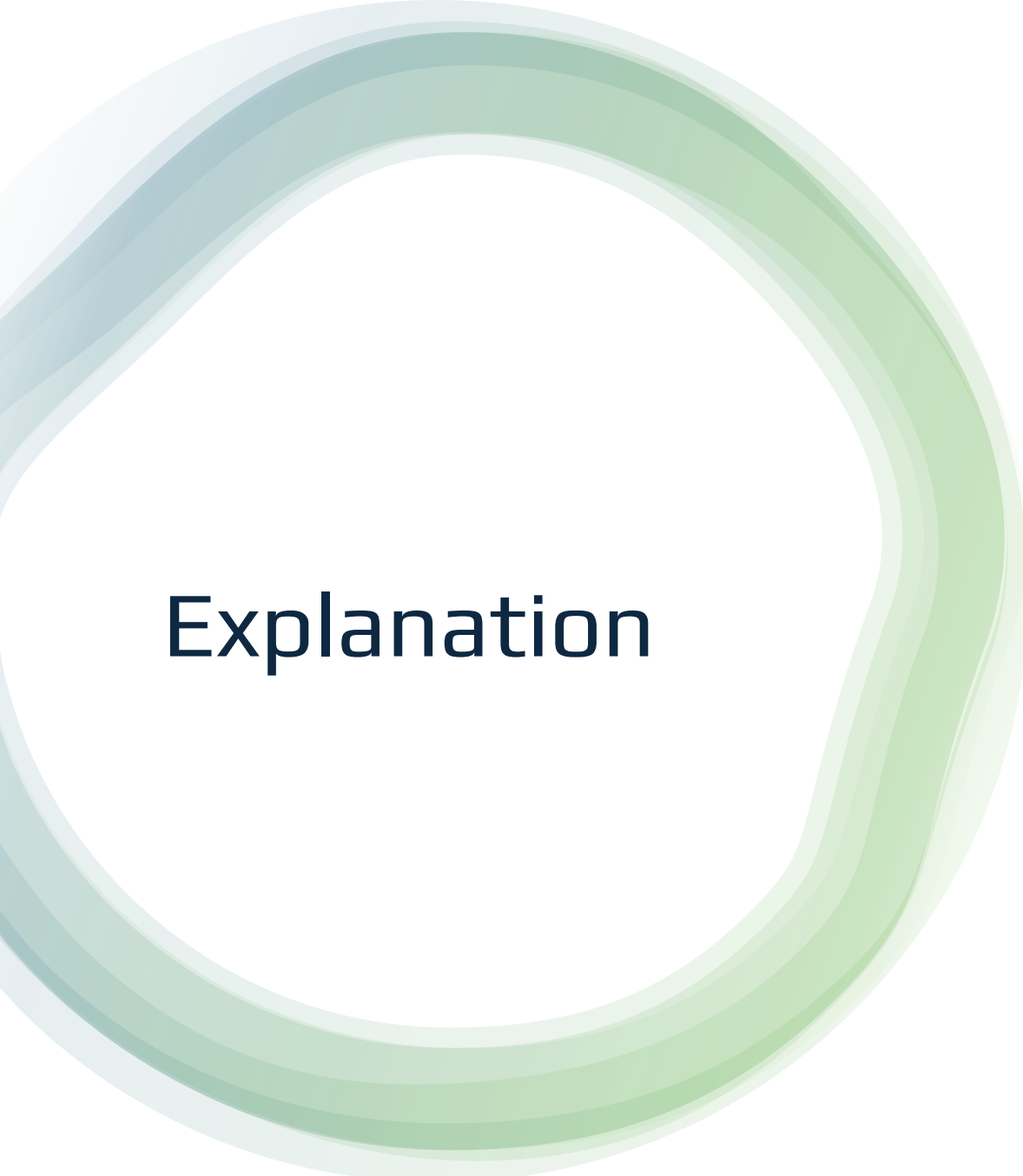
# Passing Object as Parameter in method

```
class Add {
    int a;
    int b;

    Add(int x, int y) // parametrized constructor
    {
        a = x;
        b = y;
    }

    void sum(Add A1) // object 'A1' passed as parameter in function 'sum'
    {
        int sum1 = A1.a + A1.b;
        System.out.println("Sum of a and b :" + sum1);
    }
}

public class Main {
    public static void main(String arg[]) {
        Add A = new Add(5, 8);
        /* Calls the parametrized constructor
        with set of parameters*/
        A.sum(A);
    }
}
```



# Explanation

- While creating a variable of class type, we only create a reference to an object.
- When we pass this reference to a function, the parameters that receive it will refer to the same object as that referred to by the argument.



## Passing Object as Parameter in Constructor

- One of the most common uses of objects as parameters involves constructors. A constructor creates a new object initially the same as passed object. It is also used to initialize private members.
- 

# Passing Object as Parameter in Constructor

Copy

```
class Add {  
    private int a, b;  
  
    Add(Add A) {  
        a = A.a;  
        b = A.b;  
    }  
  
    Add(int x, int y) {  
        a = x;  
        b = y;  
    }  
  
    void sum() {  
        int sum1 = a + b;  
        System.out.println("Sum of a and b :" + sum1);  
    }  
}  
  
public class Main {  
    public static void main(String arg[]) {  
        Add A = new Add(15, 8);  
        Add A1 = new Add(A);  
        A1.sum();  
    }  
}
```





# Returning the Object from method

- In java, a method can return any type of data, including class type objects.
- For ex: In the program given below, the **add()** method return an object which contain sum of values of two different Numbers(objects).

```
import java.util.Scanner;

class TwoNum {
    private int a, b;
    Scanner kb = new Scanner(System.in);

    void getValues() // getValues() take values of a,b for every no.
    {
        System.out.print("Enter a: ");
        a = kb.nextInt();
        System.out.print("Enter b: ");
        b = kb.nextInt();
    }

    void putValues() // putValues() show values for every no.
    {
        System.out.println(a + " " + b);
    }

    TwoNum add(TwoNum B) /*class type function add() take object 'B' as parameter*/ {
        TwoNum D = new TwoNum(); //object D act as instance variable
        D.a = a + B.a;
        D.b = b + B.b;
        return (D); //returning object D
    }
}
```

```
}  
}  
  
public class Main {  
    public static void main(String arg[]) {  
        TwoNum A = new TwoNum();  
        A.getValues();  
        A.putValues();  
  
        TwoNum B = new TwoNum();  
        B.getValues();  
        B.putValues();  
  
        TwoNum C;  
        /*object A calls add() passing object B  
        as parameter and result are return at C*/  
        C = A.add(B);  
  
        C.putValues();  
    }  
}
```

# References

- <https://medium.com/@asli.beyhann/pass-by-value-and-pass-by-reference-in-java-an-in-depth-exploration-9c0e4534721e>
- [includehelp.com/java/object-as-an-argument-in-java.aspx](http://includehelp.com/java/object-as-an-argument-in-java.aspx)